Lab 4 Reflection — Decorator Pattern

Riejed Aniko L. Macatula | BSIT 3A | Integrative Programming and Technologies

The smell

The without-decorator version starts with BaseHttpClient, then LoggingHttpClient extends BaseHttpClient, then CachingHttpClient extends BaseHttpClient, and finally LoggingCachingHttpClient extends LoggingHttpClient, which has to re-declare its own $cache array and copy the entire body of both parent classes just to get logging and caching in one object. That last class is the tell: subclassing can only combine two behaviors by writing a third class that duplicates both. With logging, caching, retry, and token counting as four independent features, inheritance needs a class for every combination someone might want, 2^4 = 16 classes for behavior that's really just wrapping a call and doing something before or after.

The fix

Decorator replaces the subclass explosion with composition. HttpClient is the shared interface, BaseHttpClient is the real implementation, and HttpDecorator is an abstract class holding a wrapped HttpClient while implementing the same interface. Each concrete decorator (Logging, Caching, Retry, TokenCounter) wraps whatever it's given and adds one behavior around the call to this->wrapped->get(url). RetryDecorator::get() loops up to three times and only rethrows the exception on the last attempt; TokenCounterDecorator::get() calls the wrapped client once and echoes a word count on the result. Neither decorator needs to know what's inside it or what will wrap it next.

Stacking is just constructor nesting: new Retry(new TokenCounter(new Caching(new Logging(new BaseHttpClient())))). Four classes can produce any of the sixteen combinations without a fifth class being written. Order matters, and I could see it directly in the output. Wrapping Caching around Logging means a cache hit short-circuits before Logging ever runs, so the second call prints no log lines at all. Flipping it to Logging around Caching logs on every call, cache hit or not, since the log wraps the cache check instead of the other way around. Same four decorators, same four classes, different runtime behavior depending purely on wrapping order.

Gamma citation

Gamma et al. (1994, p. 175) define Decorator as a way to "attach additional responsibilities to an object dynamically" as "a flexible alternative to subclassing for extending functionality." That's the whole lab: Logging, Caching, Retry, and TokenCounter all attach behavior to an HttpClient at runtime through composition, and no class needed to be written just to combine two of them.